

```
Void Consumer (Void)
```

```
{
  int itemc;
```

```
  While (true)
```

```
  {
    while (Count < n)
```

```
    itemc =
```

```
    Out
```

1. Load $R_c, m[Count]$
2. DECR R_c
3. Store $m[Count]$

$n=8$
Buffer[0...n-1]



Count

```
int Count=0;
```

```
Void Producer(Void)
```

```
{
  int itemp;
```

```
  while (true)
```

```
  {
    Produce_item(itemp);
```

```
    While (Count == n);
```

```
    Buffer[in] = itemp;
```

```
    in = (in+1) mod n;
```

```
    Count = Count + 1;
```

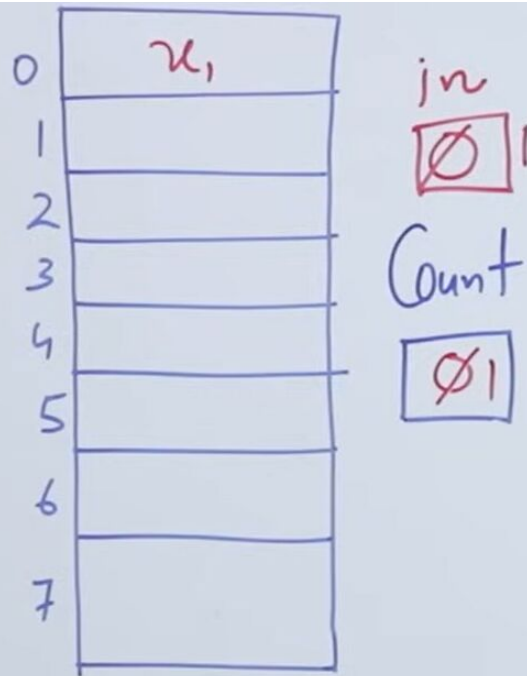
- Load $R_p, m[Count]$;
- INCR R_p ;
- Store $m[Count], R_p$;

Now void producer (void).... Int item p
this is the local variable of producer

```

itemc;
while (true)
    Out
    while (count == 0);
    itemc = Buffer(out);
    Out = (Out + 1) mod n;
    Count = Count - 1;
    Process_item(itemc);

```



Case I. x_1
 $(0+1) \bmod 8$
 $1 \bmod 8 = 1$

```

void producer()
{
    int itemp;
    while (true)
    {
        produce_item(itemp);
        while (count == n); // Buffer full
        Buffer[in] = itemp;
        in = (in + 1) mod n;
        Count = Count + 1;
        {
            load Rp, m[count];
            INCR Rp;
            Store m[count], Rp;
        }
    }
}

```

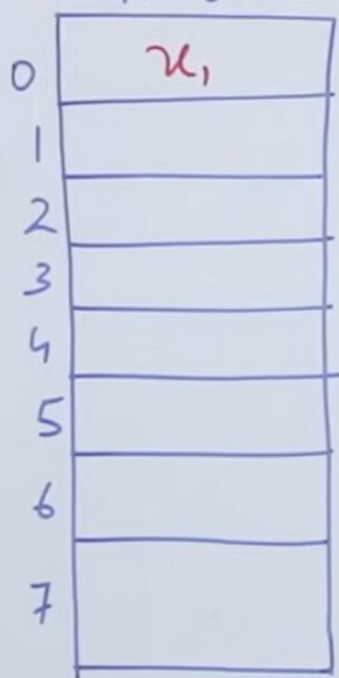
Let's say how the consumer works

```

{
  int item;
  while (true)
  {
    (count == 0);
    C = Buffer(out);
    out = (out + 1) mod n;
    count = count - 1;
    process_item(item);
  }
}

```

Out



in $\emptyset$ |

Count $\emptyset$ |

Case I. x_1

$(0+1) \bmod 8$

$1 \bmod 8 = 1$

Now whenever consumer wants to consume an item

```

void producer(n)
{
  int item;
  while (true)
  {
    produce_item(item);
    while (count == n);
    Buffer[in] = item;
    in = (in + 1) mod n;
    Count = Count + 1;
  }
}

```

→ Buffer full

load R_p , r
INCR R_p
Store m[u]

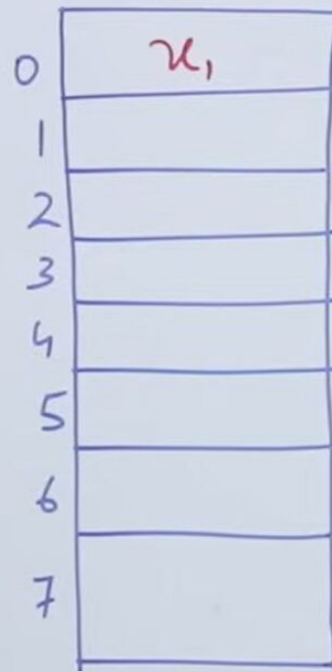

```

{
  int itemc;
  while (true)
  {
    while (count == 0);
    itemc = Buffer(out);
    Out = (Out + 1) mod n;
    Count = Count - 1;
    process_item(itemc);
  }
}

```

Out

d R_p (count) #], R_c



in Ø

Count Ø

Case I. x₁

$(0+1) \bmod 8$

$1 \bmod 8 = 1$

Now whenever consumer
wants to consume an item

```

void producer(n)
{
  int itemp;
  while (true)
  {
    produce_item(itemp);
    While (count == n);
    Buffer[in] = itemp;
    in = (in + 1) mod n;
    Count = Count + 1;
  }
}

```

load R_p
INCR
Store m

```

int itemc;
while (true)

```

Out

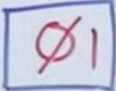

```

while (count == 0);
itemc = Buffer(out);
} Out = (Out + 1) mod n;
Count = Count - 1;
} Process_item(itemc);
}

```



in

Count


Case I. x₁
 $(0+1) \bmod 8$
 $1 \bmod 8 = 1$

void producer(n)

```

{
int itemp;
while (true)

```

```

{
produce_item(itemp);

```

```

} while (count == n);
Buffer[in] = itemp;

```

→ Buffer full

```

in = (in + 1) mod n;

```

```

Count = Count + 1;

```

```

}
}

```

```

load Rp, m[count];
INCR Rp;
Store m[count], Rp;

```

Void Consumer (Void)

int itemc;

While (true)

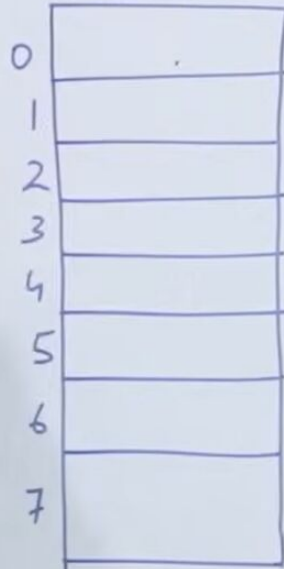
While (count == 0)

itemc = Buffer

Out = (Out + 1) % n;

Out
1

Buffer[0... n-1]



in
0

Count

0

int Count = 0;

Void Producer (Void)

{
int itemp;

While (true)

{
Produce - item(itemp);

While (Count == n);

→ Buffer full

Buffer[in] = itemp;

in = (in + 1) % n;

Count = Count + 1;

load Rp, m[Count]
INCR Rp;
Store m[Count], Rp

Case I. x_1

$(0+1) \bmod 8$
 $1 \bmod 8 = 1$

$(0+1) \bmod 8$
 $1 \bmod 8 = 1$

we took out the X1 that we produced


```
Void Consumer (Void)
```

```
{
  int itemc;
  While (true)
```

Buffer
Empty

```
While (Count == 0);
```

```
itemc = Buffer(Out);
```

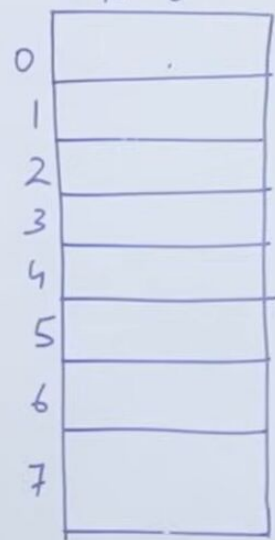
```
Out = (Out + 1) mod n;
```

```
Count = Count - 1;
```

```
Produce_item(itemc);
```

Out
[0]

n=8
Buffer[0...n-1]



in
[0]

Count
[0]

```
int Count=0;
```

```
Void Producer(Void)
```

```
{
  int itemp;
  While (true)
```

```
{ Produce_item(itemp);
```

```
While (Count == n);
```

```
Buffer[in] = itemp;
```

```
in = (in + 1) mod n;
```

```
Count = Count + 1;
```

Buffer
full

Case I. x_1

$(0+1) \bmod 8$
 $1 \bmod 8 = 1$

$(0+1) \bmod 8$
 $1 \bmod 8 = 1$

this will be perfectly fine, but
now we are moving towards the next

Void Consumer (Void)

```
{
  int itemc;
  while (true)
```

Buffer Empty

While (Count == 0);

itemc = Buffer(Out);

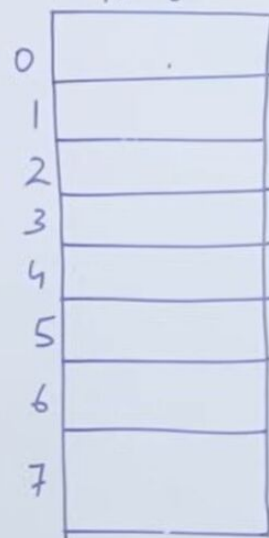
Out = (Out + 1) mod n;

Count = Count - 1;

Process_item(itemc);

Out
[0]

n=8
Buffer[0....n-1]



in
[0]

Count

[0]

int Count=0;

Void Producer(Void)

```
{
  int itemp;
  while (true)
```

{ Produce_item(itemp);

While (Count == n);

Buffer full

Buffer[in] = itemp;

in = (in + 1) mod n;

Count = Count + 1;

load Rp, m[Count];
INCR Rp;
Store m[Count], Rp;

Let's say... now we are talking about process synchronization


```
int itemc;
while (true)
```

Out
~~0~~ 1

```
while (count == 0);
```

```
itemc = Buffer(out);
```

```
out = (out + 1) mod n;
```

```
count = count - 1;
```

```
process_item(itemc);
```

$R_c = 32$

Buffer[0...n-1]

0	
1	x_2
2	x_3
3	x_4
4	
5	
6	
7	

In
~~3~~ 4

Count

~~3~~ 4

Void Producer(Void)

```
{
  int itemp;
```

```
  while (true)
```

```
{
  produce_item(itemp);
```

```
  while (count == n);
```

Buffer full

```
  Buffer[in] = itemp;
```

```
  in = (in + 1) mod n;
```

```
  count = count + 1;
```

Terminate

1. Load $R_p, m[count];$
2. INCR $R_p;$
3. Store $m[count], R_p;$

Case. Producers.

I_1, I_2 Consumers I_1, I_2 Producer I_3

R_p
~~3~~ 4

```
int itemc;
while (true)
```

Out
~~0~~ 1

```
while (count == 0);
```

```
itemc = Buffer(out);
```

```
out = (out + 1) mod n;
```

```
count = count - 1;
```

```
process_item(itemc);
```

$R_c = 32$

Buffer[0...n-1]

0	
1	x_2
2	x_3
3	x_4
4	
5	
6	
7	

In
~~3~~ 4

Count

~~3~~ 4

Void producer(void)

```
{
int itemp;
```

```
while (true)
```

```
{
produce_item(itemp);
```

```
while (count == n);
```

Buffer full

```
Buffer[in] = itemp;
```

```
in = (in + 1) mod n;
```

```
count = count + 1;
```

Terminate

1. load $R_p, m[count];$
2. INCR $R_p;$
3. Store $m[count], R_p;$

Case. Producers.

I_1, I_2 Consumers

Producer I_3

when it got terminated then the consumer will get the control back

Void Consumer (Void)

itemc;
while (true)

Out
~~0~~ 1

while (count == 0);

itemc = Buffer(Out);

Out = (Out + 1) mod n;

Count = Count - 1;

Process_item(itemc);

Rp = 32

n=8
Buffer[0...n-1]

0	
1	x ₂
2	x ₃
3	x ₄
4	
5	
6	
7	

In
~~3~~ 4

Count

~~3~~ 4 2

int Count=0;

Void Producer(Void)

{
int itemp;

while (true)

{
Produce_item(itemp);

while (Count == n);

Buffer full

Buffer[in] = itemp;

in = (in + 1) mod n;

Count = Count + 1;

Terminate

1. Load Rp, m[Count];
2. INCR Rp;
3. Store m[Count], Rp;

x₄
Producers

Consumer I₁, I₂

Consumer I₁, I₂

Producer I₃ Consumer I₃

Void Consumer (Void)

itemc;
while (true)

Out
~~0~~ 1

while (count == 0);
itemc = Buffer(out);

Out = (Out + 1) mod n;

Count = Count - 1;

Process_item(itemc);

$R_c = 32$

$n=8$
Buffer[0...n-1]

0	
1	x_2
2	x_3
3	x_4
4	
5	
6	
7	

In
~~3~~ 4

Count
~~3~~ 4 2

int Count=0;
Void Producer(Void)

{
int itemp;
while (true)
{
produce_item(itemp);

while (count == n);

Buffer full

Buffer[in] = itemp;

in = (in + 1) mod n;

Count = Count + 1;
1. Load $R_p, m[count]$;
2. INCR R_p ;
3. Store $m[count], R_p$;

Terminate

Case Producer.

$R_p = 34$

I_1, I_2 Consumer

I_1, I_2 Consumer

Producer I_3 Consumer I_3

Void Consumer (Void)

{

int C;

while (true)

if (Count == 0);

int C = Buffer(Out);

(Out++) mod n;

Out--;

Print(C);

Out
01

n=8

Buffer[0...n-1]

0	
1	x ₂
2	x ₃
3	x ₄
4	
5	
6	
7	

In
34

Count

342

int Count=0;

Void Producer(Void)

{

int itemp;

while (true)

{

Produce-item(itemp);

while (Count == n);

Buffer[in] = itemp;

in = (in+1) mod n;

Count = Count + 1;

Terminate

1. Load R_p, m[Count];

2. INCR R_p;

3. Store m[Count], R_p;

Buffer full

Call Producer.

Producer I, I₂

Consumer I, I₂

Producer I₃

Consumer I₃

x₄

R_p
34

3

Void Consumer (Void)

```
{
  int itemc;
  While (true)
```

Out
~~0~~ 1

Buffer Empty

While (Count == 0);

itemc = Buffer(Out);

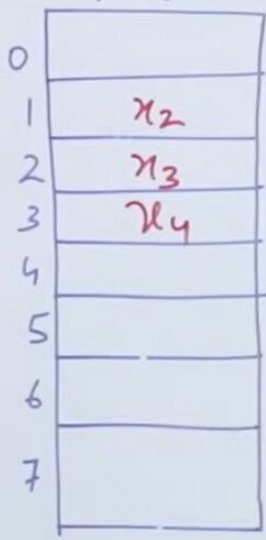
Out = (Out + 1) mod n;

Count = Count - 1;

Process_item(itemc);

Rc = 32

n=8
Buffer[0...n-1]



In
~~3~~ 4

Count

~~3~~ 4 2

int Count=0;

Void Producer(Void)

```
{
  int itemp;
  While (true)
```

{ Produce_item(itemp);

While (Count == n);

Buffer full

Buffer[in] = itemp;

in = (in + 1) mod n;

Count = Count + 1;

1. Load Rp, m[Count];
2. INCR Rp;
3. Store m[Count], Rp;

Rp
~~3~~ 4

Call Producers.
Producer I1, I2 Consumer I1, I2

Terminate
Producer I3 Consumer I3

Void Consumer (Void)

```
{
  int itemc;
  While (true)
```

Out
~~1~~

Buffer
Empty

```
  While (Count == 0);
  itemc = Buffer(Out);
```

```
  Out = (Out + 1) mod n;
```

```
  Count = Count - 1;
```

```
  Process_item(itemc);
```

$R_c = 32$
Terminate

$n=8$
Buffer[0...n-1]

0	
1	x_2
2	x_3
3	x_4
4	
5	
6	
7	

In
~~3~~ 4

Count
~~3~~ 4 2

int Count=0;
Void Producer(Void)

```
{
  int itemp;
  While (true)
  {
    Produce_item(itemp);
```

```
  While (Count == n);
  Buffer[in] = itemp;
```

Buffer
full

```
  in = (in + 1) mod n;
```

```
  Count = Count + 1;
```

```
  1. Load  $R_p, m[Count]$ ;
  2. INCR  $R_p$ ;
  3. Store  $m[Count], R_p$ ;
```

R_p
~~3~~ 4

x_4
Call Producer.

Producer I_1, I_2 Consumer I_1, I_2

Terminate

Producer I_3 Consumer I_3

Void Consumer (Void)

```
{
    int item;
    while (true)
    {
```

```
        if (count == 0)
        {
            item = Buffer(out);
            out = (out + 1) % n;
        }
```

Out
1

n=8
Buffer[0...n-1]

0	
1	x ₂
2	x ₃
3	x ₄
4	
5	
6	
7	

In
3 4

Count
3 4 2

int Count=0;
Void Producer(Void)

```
{
    int item;
    while (true)
    {
        produce_item(item);
```

```
        while (count == n)
        {
            Buffer[in] = item;
```

Buffer full

in = (in + 1) % n;

```
        count = count + 1;
        1. Load Rp, m[count];
        2. INCR Rp;
        3. Store m[count], Rp;
```

R_p
3 4

Call Producer.

Producer I₁, I₂

Consumer I₁, I₂

Producer I₃ Consumer I₃

Void Consumer (Void)

{

int

W

Buffer
Empty

{

W

while

(

Count

== 0)

;

Buffer

(out);

Count

++;

}

}

}

}

}

}

}

Out
1

== 0);

Buffer(out);

Count

++;

}

}

}

}

}

}

}

}

}

}

}

}

}

n=8

Buffer[0...n-1]

0	
1	x ₂
2	x ₃
3	x ₄
4	
5	
6	
7	

In
3 4

Count

3 4 2

int Count=0;

Void Producer(Void)

{

int itemp;

while(true)

{

produce_item(itemp);

while(count == n);

Buffer[in] = itemp;

in = (in+1) mod n;

Count = Count + 1;

Terminate

1. Load R_p, m[Count];
2. INCR R_p;
3. Store m[Count], R_p;

Buffer full

R_p
3 4

x₄

Producers

Consumer I₁, I₂

Producer I₃ Consumer I₃

Void Consumer (Void)

```
{
  int itemc;
  While (true)
```

Out
1

Buffer
Empty

While (Count == 0);

itemc = Buffer(Out);

Out = (Out + 1) mod n;

Count = Count - 1;

Process item(itemc);

Rc = 32

n=8
Buffer[0...n-1]

0	
1	x ₂
2	x ₃
3	x ₄
4	
5	
6	
7	

In
34

Count

34

int Count=0;

Void Producer(Void)

```
{
  int itemp;
  While (true)
```

Produce item(itemp);

While (Count == n);

Buffer
full

Buffer[in] = itemp;

in = (in + 1) mod n;

Count = Count + 1;

Terminate

1. Load R_p, m[Count];
2. INCR R_p;
3. Store m[Count], R_p;

Call Producer.

Producer I₁, I₂ Consumer I₁, I₂

Producer I₃ Consumer I₃